RESQNET + KAVACH

Unified Real-Time Disaster Coordination Platform — Complete Project Description

SMART INDIA HACKATHON

Problem Statement (as addressed)

> *During floods / cyclones / landslides, delayed information flow between citizens, NDRF, and local administration causes poor resource allocation and slower rescue response.*

The core failure in Indian disaster response is not a lack of resources — it is a **coordination gap**. Three groups act on three different, lagging pictures of the same event:

- **Citizens** know exactly what is happening at their location but have no structured channel to report it — they call overloaded helplines or post on social media.
- **Local administration / District Disaster Management Authority** receives fragmented phone reports and paper registers; they cannot see where the need is concentrated or which team is closest.
- **NDRF / rescue teams** are dispatched on verbal instructions, often to the wrong place or in the wrong order, while other clusters wait.

Meanwhile **IMD / SACHET official warnings** live in a separate government portal that neither citizens nor field teams routinely watch.

RESQNET + KAVACH closes this loop. It is a single map-based coordination platform with two live feeds layered on one authority dashboard — **geo-tagged citizen reports** (pin + severity + description) and **available resources** (shelters, rescue teams, ambulances, supply stock, each with a capacity / status field) — plus a **real-time allocation engine** that tells the operator which unit should move to which incident cluster, in what order, and updates the map live as new reports arrive.

1. How It Addresses the Problem

Gap in current response	What RESQNET + KAVACH does
Citizens have no structured reporting channel	**KAVACH app**: one-tap SOS, AI SMS chat, and an AI voice helpline (IVR). Every report is geo-tagged, carries photo / text, and is parsed into structured facts (people count, injured, trapped, needs, vulnerable persons).
No-internet zones during disasters	**SMS decision-tree + IVR voice** paths work with zero data. The AI helpline runs a full offline scripted flow when the backend is unreachable; triage still works on deterministic rules alone.
Authority has no single operational picture	**RESQNET dashboard**: one Azure Maps view with citizen-report pins (colour-coded by AI-computed severity), discovered resources, live caller GPS, and official SACHET / IMD warning polygons — all clipped to the operator's district.
Official IMD / SACHET alerts sit in a separate silo	The dashboard pulls the **live SACHET CAP feed** (via a proxy), draws the affected zones as map polygons, and lets the operator fire an area-targeted broadcast that reaches citizens inside KAVACH.
Resources dispatched ad-hoc, wrong order, wrong place	**Allocation engine**: clusters related reports, ranks them in **strict severity order** (every CRITICAL before any HIGH before any MEDIUM), matches each cluster's specific needs to compatible resource types, spreads the load across facilities, and **depletes availability** as units are committed so the next decision only sees what is genuinely free. Shown live on the map as animated routes.
Citizen never knows if help is coming	KAVACH shows a **4-stage request tracker** (Received → Assigned → En route → Resolved) that updates the instant the operator changes status.

The judged core — the allocation logic — goes well beyond nearest-available matching

1. **Cluster** unresolved incidents (within 2 km, within a 6-hour window, sharing a need).
2. **Score urgency (0–100)** per cluster with a trained linear model over people affected, injured, trapped, aggregate demand, time elapsed, and the deterministic priority score.
3. **Order the queue by severity tier first** — a large MEDIUM incident can never overtake a CRITICAL one; the urgency score only breaks ties *inside* a tier.
4. **Decompose each cluster into per-need requirements** — injured people drive *medical* demand, trapped people drive *rescue* demand, the headcount drives *relief* demand. A cluster needing both rescue and medical gets an NDRF team **and** an ambulance, not two of whichever is closest.

5. **Rank candidate resources** by `suitability (distance + ETA) × reservation (scarcity vs urgency) × spread (penalty for units already committed this run)`.
6. **Greedily assign units** until the cluster's people are covered, deducting each unit from the running availability pool.
7. **Commit** — availability is decremented in the database, incidents move to ASSIGNED, and the plan is pushed to citizens.

In the working prototype (Kanpur Nagar, 5 live incidents, 67 discovered facilities): CRITICAL clusters were served first, 7 units dispatched to **7 different facilities**, the district's "Resources Available" counter dropped 67 → 60, and where the district had no dedicated water tanker or shelter unit, the nearest available team was substituted with an explicit warning to the operator.

2. Innovation and Uniqueness

A. Deterministic core, AI at the edges — fully auditable decisions

The LLM only *reads and phrases*. It never decides severity, priority, or allocation. Every operational number is produced by explicit rule-based code with a human-readable reason list ("Injury is reported", "Vulnerable person present", "No dedicated water unit in this district — nearest medical team substituted"). An authority can defend or override every recommendation — critical for government adoption where a black-box dispatch decision is unacceptable.

B. Severity as a hard constraint, not a soft score

Most allocation demos rank incidents by a single blended score, which lets a high-headcount low-severity incident jump the queue. RESQNET enforces strict tier ordering (CRITICAL → HIGH → MEDIUM → LOW); the learned model only discriminates *within* a tier. This mirrors how a real control room triages.

C. Need-aware, not headcount-aware, matching

The allocator sends the *right kind* of unit for each requirement family and spreads load across facilities, instead of dividing a headcount by capacity and sending everything to the single nearest depot.

D. Live, depleting resource state

Committing an allocation actually decrements `available_units` in the database and reassigns incidents, so a second allocation run sees a genuinely smaller pool. The map shows the district's capacity shrinking in real time — the coordination value made visible.

E. Trained triage model grounded in doctrine, not hand-tuned numbers

The urgency model is trained by gradient descent on synthetic scenarios labelled by a rule-based "expert" function encoding real triage doctrine — START mass-casualty triage, the trauma golden hour, the search-and-rescue 36-hour entrapment window. Doctrine-derived interaction features (entrapment as a *proportion* of the group, time-pressure on two different horizons, trapped-AND-injured compounding) let a transparent linear model approximate non-linear expert judgement: **test error dropped ~3× (MAE 11.1 → 3.7 on a 0–100 scale)**, and the model independently learned to weight injury / entrapment *ratios* over raw counts. The pipeline is built so real logged incident-outcome data becomes a drop-in replacement for the synthetic labels.

F. True multi-channel, multilingual intake with graceful degradation

App, SMS decision-tree, and AI voice helpline — all in English / Hindi / Hinglish, with browser speech-to-text and text-to-speech. Every remote dependency (LLM, SACHET proxy, maps API, backend) has a local fallback, so the platform keeps working under exactly the network stress a disaster creates.

G. Official + crowd data fused on one canvas

IMD / SACHET CAP warnings and citizen reports on the same district-scoped map, with a hazard-specific advisory library (flood, cyclone, earthquake, landslide, heatwave, thunderstorm, fire, tsunami, cold wave) — because "stay indoors" is right for a cyclone and dangerous for an earthquake.

3. Technologies Used

Frontend — RESQNET dashboard + KAVACH citizen app

Category	Technology
Core	HTML5, **vanilla JavaScript** (ES modules + classic scripts), no build step
Styling	**Tailwind CSS**, Font Awesome, Public Sans / Inter; custom token-based **dark mode**
Maps	**Azure Maps SDK v3** (live incident map, heatmap, alert polygons), **Google Maps JS API** (allocation-simulation map, facility discovery via Places)
Realtime client	**Firebase Web SDK v10** (Cloud Firestore live listeners)
Device APIs	Web Speech API (STT `webkitSpeechRecognition`, TTS `speechSynthesis`, hi-IN / en-IN), Geolocation `watchPosition` (live caller GPS), Web Audio (siren), `localStorage`, `BroadcastChannel`

Backend — Citizen Intelligence API

Category	Technology
Runtime	**Python 3**, **FastAPI**, **Uvicorn**, **Pydantic**
Deployment	**Google Cloud Run** — containerised (Docker), built by Cloud Build, autoscaling, scale-to-zero, HTTPS endpoint
Secrets	**GCP Secret Manager** (Firebase Admin service-account key)
DB access	**firebase-admin** (Firestore Admin SDK)

AI / ML

Purpose	Technology
Citizen-message understanding (Hindi / English / Hinglish, SMS shorthand)	**Qwen** `qwen3:4b` **via Ollama** (dev) → deterministic rule engine fallback (always available)
AI voice helpline phrasing	**Groq** (`llama-3.1-8b-instant`) → **Google Gemini** (`gemini-2.0-flash`) → fixed multilingual script
Severity & priority	Deterministic additive rule engine (no LLM)
Incident clustering	Union-find over geographic + temporal + semantic proximity
Resource allocation	Custom **linear urgency model**, trained by batch gradient descent on doctrine-labelled synthetic scenarios (no heavy ML library); need→type compatibility matching; distance / ETA + scarcity + spread scoring
Routing / ETA	**Google Distance Matrix API**, with a haversine estimate (×1.3 @ 30 km/h) fallback
Voice transcription (optional)	faster-whisper (lazy-loaded, for the audio-upload endpoint)

Infrastructure

Piece	Platform
Static site (both apps)	**Firebase Hosting** (global CDN)
Real-time database & message bus	**Cloud Firestore**
AI backend	**Google Cloud Run** + Cloud Build + Artifact Registry
SACHET / IMD alert proxy	**Cloudflare Workers** (edge serverless, CORS + header injection, 120 s edge cache)
Containerisation	Docker
Source control	Git / GitHub

External data sources

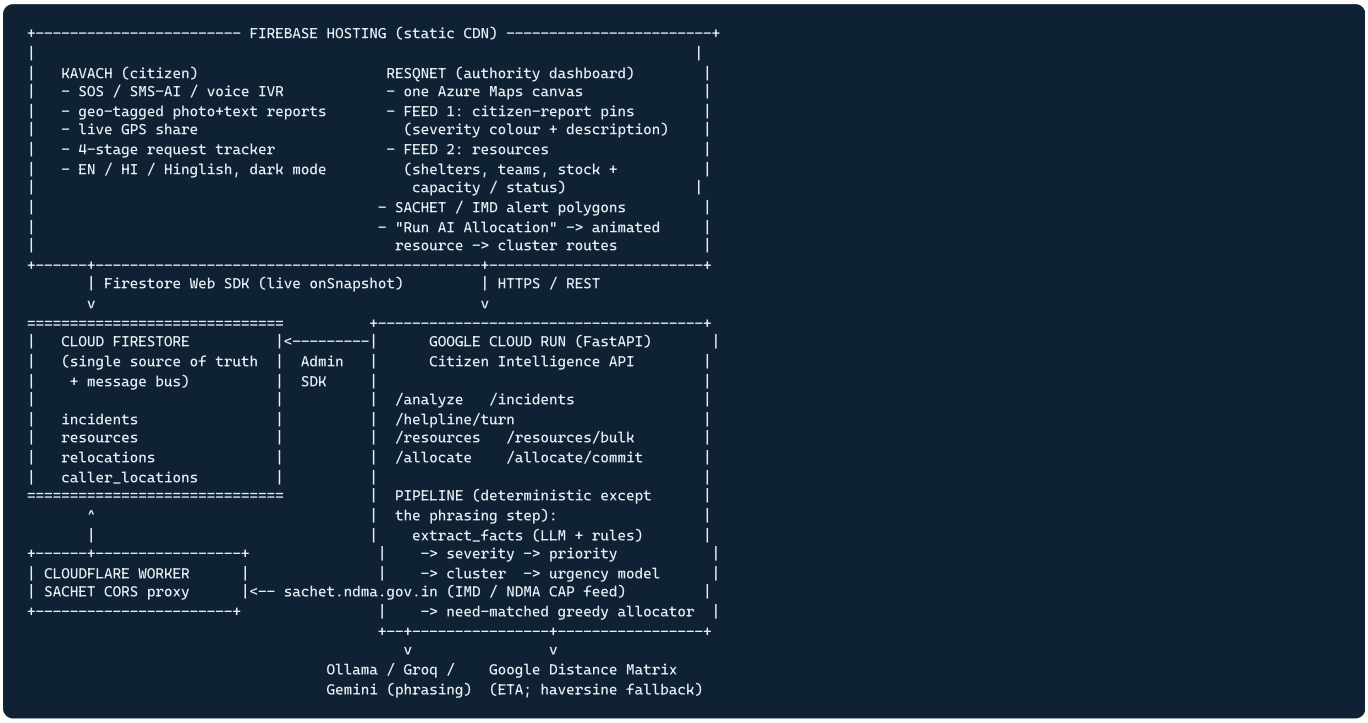
SACHET / NDMA CAP alert feed · IMD warnings · Nominatim (district boundaries) · Photon (Komoot) + Overpass (facility discovery — hospitals, police, fire stations) · Open-Meteo (weather + forecast).

Hardware

No custom hardware. Runs on any smartphone browser (citizen) and any desktop browser (control room). Server side is fully serverless. The **SMS / IVR fallback** is designed to run over a telecom SMS gateway / IVR line in production (simulated in the prototype).

4. Methodology and Implementation Process

4.1 System architecture



4.2 End-to-end data flow (the coordination loop)

```
1. REPORT
   Citizen -> KAVACH -> SOS / SMS-AI / voice helpline
   e.g. "Ghar mein paani bhar gaya, hum 6 log hain, papa injured, dadi chal nahi sakti"
        (v) geo-tagged, photo/text, POST /incidents   (SMS / IVR path if no data)

2. UNDERSTAND  (Cloud Run)
   LLM drafts structured facts -> deterministic rules validate / re-derive every field
   -> needs = [rescue, medical], people = 6, injured = true,
      vulnerable = [elderly, mobility_impaired], env = [water_entered_house]

3. SCORE  (no LLM)
   severity features -> priority 0-100 + level + reason list  -> CRITICAL

4. STORE & SYNC
   Firestore incidents/{id}  -> live push to every RESQNET dashboard
   KAVACH starts streaming caller GPS to caller_locations/{id}

5. VISUALISE  (RESQNET)
   red pin on the map, row on the Dispatch Board (priority + age + need tags),
   pulsing caller-GPS marker -- all clipped to the operator's district polygon

6. CLUSTER + ALLOCATE  (Cloud Run  /allocate)
   group related reports -> urgency score per cluster
   -> order queue CRITICAL -> HIGH -> MEDIUM -> LOW
   -> per-need requirements -> rank resources (distance + ETA + scarcity + spread)
   -> greedily assign units, deduct from availability pool
   -> recommendations with routes, ETAs, reasons

7. COMMIT + DISPATCH  (RESQNET  /allocate/commit)
   units animate along routes on the map, tier by tier
   available_units decremented in Firestore, incidents -> ASSIGNED
   "Resources Available: 67 -> 60"

8. CLOSE THE LOOP  (KAVACH)
   citizen's request tracker advances "Assigned -> En route -> Resolved"
   toast on every status change
   resolved incidents sink to the bottom of the dashboard board and the citizen list

PARALLEL: RESQNET -> Cloudflare Worker -> SACHET CAP feed -> alert polygons on the map
          -> operator fires an area-targeted broadcast -> hazard-specific advisory to KAVACH
```

4.3 Implementation phases

Phase	Deliverable	Status
1	Citizen intake — KAVACH app: geo-tagged SOS, SMS decision-tree, voice IVR helpline, photo / text	**Working prototype**
2	AI understanding — LLM + deterministic fact extraction, severity, priority (Hindi / English / Hinglish)	**Working, tested** (offline fallback verified)
3	Authority dashboard — Azure map, two live feeds, district gate, SACHET overlay, dispatch board	**Working prototype**
4	Resource discovery — auto-import real district facilities (Photon / Overpass / Places), capacity / status fields	**Working** (67 real facilities imported for Kanpur Nagar in testing)
5	Allocation engine — clustering, trained urgency model, severity-tiered need-matched greedy allocation, availability depletion, commit	**Working, tested end-to-end**
6	Deployment — Firebase Hosting + Cloud Run + Cloudflare Worker + Firestore	Wired; Cloud Run deploy in progress
7	Production SMS / IVR — integrate a telecom SMS gateway + IVR line behind the existing endpoints	Planned (endpoints already source-agnostic)

4.4 Working prototype evidence

- KAVACH tested in-browser: citizen registered in **Kanpur Nagar, UP**; the location card correctly stays "Kanpur Nagar" (previously reverted to a hardcoded default); the hazard classifier correctly maps CAP event strings → flood / cyclone / earthquake / landslide / heatwave / thunderstorm; the "Is everyone safe?" prompt fires only for an in-area SACHET alert and shows flood-specific vs earthquake-specific instructions; dark mode applies before first paint.
- RESQNET tested against a live local instance of the Cloud Run backend: district gate resolves to Kanpur Nagar, 67 facilities discovered and imported, **AI Allocation run animated 7 units to 7 facilities in CRITICAL → HIGH → MEDIUM → LOW order**, availability counter fell 67 → 60, substitution warnings shown where the district lacked a resource type, resolved incidents sorted to the bottom of the dispatch board.
- Backend regression test: `GET /allocate` leaves state untouched; `POST /allocate/commit` deducts availability and reassigns incidents; a second run correctly sees the depleted pool.
- Urgency model retrained: test MAE 11.12 → 3.68.

5. Feasibility Analysis

Technical feasibility — HIGH

- Every component is built on mature, free-tier-friendly managed services (Firebase, Cloud Run, Cloudflare, Firestore). No custom infrastructure to operate.
- A working prototype already demonstrates the full loop, including the judged allocation logic.
- Serverless + scale-to-zero means the platform costs almost nothing at idle and scales automatically during a disaster surge.
- The deterministic core has **no hard dependency on the LLM** — the system delivers structured triage and allocation even with the AI model completely offline, which removes the single biggest technical risk.

Operational feasibility — HIGH, with one dependency

- District facility data (hospitals, police, fire) is auto-discovered from OpenStreetMap, so a new district is live in minutes with zero manual data entry.
- The authority dashboard needs no training beyond "the red pins are urgent, press Run Allocation, adjust status" — it mirrors a physical control-room whiteboard.
- The one real-world dependency is a **telecom partnership** for the production SMS gateway and IVR line; until then the SMS / IVR path is simulated. The application endpoints are already source-agnostic, so this is an integration, not a redesign.

Economic feasibility — HIGH

- Free tiers cover a district-scale pilot end to end. Estimated cost at real state-scale load is dominated by Maps API calls and can be capped.
- No hardware procurement. No per-seat licensing.

Scalability feasibility — HIGH

- District scoping means each control room only ever processes its own polygon of data — the system scales horizontally by district, not as one national bottleneck.
- Firestore live sync removes the need for a custom real-time server.

6. Potential Challenges and Risks

#	Challenge / Risk	Impact
R1	**Connectivity collapse** in the disaster zone — towers down, no data	Citizens can't reach the app; field teams can't get updates
R2	**False / duplicate / panic reports** flooding the queue	Allocation skewed toward noise; real CRITICAL cases delayed
R3	**LLM unavailable or wrong** (hosted API down, model hallucination)	Garbled triage; misrouted resources
R4	**Stale or inaccurate resource data** — a "shelter" that's full, a team already deployed	Allocator sends units where they can't help
R5	**Map / geo API quota or outage** (Google Places, Overpass mirrors flaky)	Facility discovery thin; ETAs degrade
R6	**Authority adoption & trust** — control rooms rely on phone + paper; an opaque "AI dispatch" is a non-starter	Platform ignored during a real event
R7	**Official-alert dependency** — SACHET feed format changes or blocks the proxy	Alert layer goes blank
R8	**Data privacy** — citizen location, health status, phone numbers	Legal exposure; citizen distrust
R9	**Surge load** — thousands of simultaneous reports in the first hour	Latency, dropped writes
R10	**Language / literacy barriers** — citizen can't type, can't read the app	Under-reporting from the most vulnerable

7. Strategies for Overcoming These Challenges

Risk	Mitigation (mostly already built)
R1 Connectivity	**SMS decision-tree + IVR voice** paths need zero data. The AI helpline has a full offline scripted flow; triage runs on deterministic rules with no server. KAVACH caches the district, boundary, and last state in `localStorage`. A `BroadcastChannel` / `localStorage` bridge keeps a same-device demo working with no network.
R2 False reports	Severity is computed only from **explicitly stated facts** — "please help" alone is never escalated to a specific need. Clustering merges duplicate reports of the same event into one demand cluster. The operator has a status control on every incident and can mark a report resolved / invalid; resolved items sink out of the queue.
R3 LLM failure	The LLM output is **never trusted directly** — Python re-derives every fact from the raw text. If the model is unreachable, `extract_facts` falls back to the pure rule engine and the helpline to a fixed multilingual script. The service still returns valid triage. Import is defensive so a missing model package cannot stop the container booting.
R4 Stale resources	Every resource carries an explicit `available_units` / `status` field. **Committing an allocation decrements availability in the database**, so the next run only sees free units. Re-running discovery is an **upsert keyed on (district, type, location)** — it never resets the availability of units already dispatched. The operator can `PATCH` any resource to correct it.
R5 Map / geo outage	Facility discovery races **Photon + Overpass + Google Places** in parallel and merges; missing categories are synthesised and flagged "DERIVED". Distance / ETA falls back to a haversine estimate. The dashboard works on OSM data alone if Places is disabled.
R6 Authority trust	**Every decision is explainable** — priority and allocation come with a plain-English reason list, severity is a hard tier the operator understands, and the allocator only *recommends*: nothing dispatches until the operator presses Commit, and they can override any assignment. The UI mirrors a control-room whiteboard.
R7 SACHET dependency	The Cloudflare Worker isolates the parsing logic in one place; the client tries Worker → local proxy → public CORS proxies. Alert absence degrades to a blank layer, not a crash.
R8 Privacy	Prototype Firestore rules are open for the demo; the production plan is token auth per district control room, field-level access control, and PII minimisation (phone / health data stored only for the incident's lifetime). Data is never placed in URLs. `.gcloudignore` strips secrets from the deploy; the Admin key lives only in Secret Manager.
R9 Surge load	Serverless autoscaling (Cloud Run scales out, Firestore is managed). District scoping partitions load. The in-memory layer is a cache over Firestore, which is the durable store, so a container restart loses nothing.
R10 Language / literacy	Full **English / Hindi / Hinglish** UI, plus **voice** input and output (STT / TTS) so a non-literate citizen can complete a report by speaking. The SMS decision-tree uses single-digit replies. Hazard advisories are pre-translated.

8. Potential Impact on the Target Audience

Citizens in the disaster zone

- A working channel to report their exact situation and *know* a team has been assigned — the 4-stage tracker removes the "is anyone coming?" panic.
- Hazard-specific, life-safety-first instructions in their own language, by voice if needed.
- Works from a basic phone over SMS / IVR when data is gone.

Local administration / DDMA operators

- One screen replaces a wall of phone calls and paper registers: where the need is, what each cluster needs, which unit is closest and free, and in what order to act.
- The allocation engine turns "who do I send?" from a guess into a ranked, explained recommendation they can accept in one click or override.
- Live capacity view — they can see the district running out of ambulances before it happens.

NDRF / rescue & relief teams

- Dispatched to the right cluster, in severity order, with an ETA and a reason — not to whatever address came over the radio last.
- Load is spread across facilities instead of everything converging on one depot.

State / national disaster authorities

- District-level dashboards roll up into a consistent operational picture.
- Every dispatch decision is logged with its rationale — an auditable record for post-event review and for training the model on real outcomes.

9. Benefits of the Solution

Social

- **Faster rescue for the most urgent cases.** Strict severity ordering guarantees trapped / injured clusters are served before large but lower-acuity ones — directly targeting the "golden hour".
- **Equity of access.** Voice + SMS + multilingual intake reaches elderly, non-literate, and rural citizens who are invisible to app-only or English-only systems.
- **Reduced panic and rumour.** An official two-way channel with status tracking and hazard-correct advice replaces social-media speculation.
- **Trust in institutions.** Transparent, explainable decisions and a visible response loop rebuild citizen confidence in disaster machinery.

Economic

- **Lower response cost per life saved.** Better-ordered, better-matched dispatch means fewer wasted vehicle-hours and less duplicated effort.
- **Near-zero fixed cost.** Serverless, free-tier-friendly, no hardware, no per-seat licences — deployable by a district authority on a minimal budget.
- **Reduced secondary losses.** Faster evacuation and relief cut the economic damage that compounds when people are stranded for days.
- **Reusable asset.** The same platform serves floods, cyclones, landslides, earthquakes, heatwaves — one build, many hazards.

Environmental / operational

- **Efficient resource movement.** Spread-aware allocation and real ETAs reduce unnecessary fuel burn from misrouted convoys.
- **Better use of existing infrastructure.** Auto-discovery of real hospitals / shelters means the platform coordinates what a district already has rather than requiring new depots.
- **Data for resilience planning.** Logged incident clusters and response times show authorities where shelters, teams, or stock are chronically short — informing where to invest before the next event.

Governance

- **Auditability.** Every priority score and allocation decision is recorded with its reasons, supporting after-action review and accountability.
- **Continuous improvement.** The training pipeline is built so real incident-to-outcome data replaces the synthetic labels — the allocator gets measurably better with every disaster it assists.